

Le but de cette séquence est de créer des jeux à l'aide de pyxel.

1. Pyxel :

Pyxel est une bibliothèque graphique, dite aussi **moteur graphique**, permettant la programmation de jeux en 2D **rétro** :



Vous pourrez retrouver une petite vidéo [ici](#)

La bibliothèque est aussi bien utilisée en programmation **fonctionnelle** (ce que l'on voit en Première NSI), qu'en programmation **objet** (ce que l'on voit en Terminale NSI).

Un moteur graphique est un puissant ensemble de composants modélisant de la physique, des actions utilisateurs, de la géométrie, du visuel, des sons ...

Tous les jeux vidéo utilisent un moteur graphique pour fonctionner.

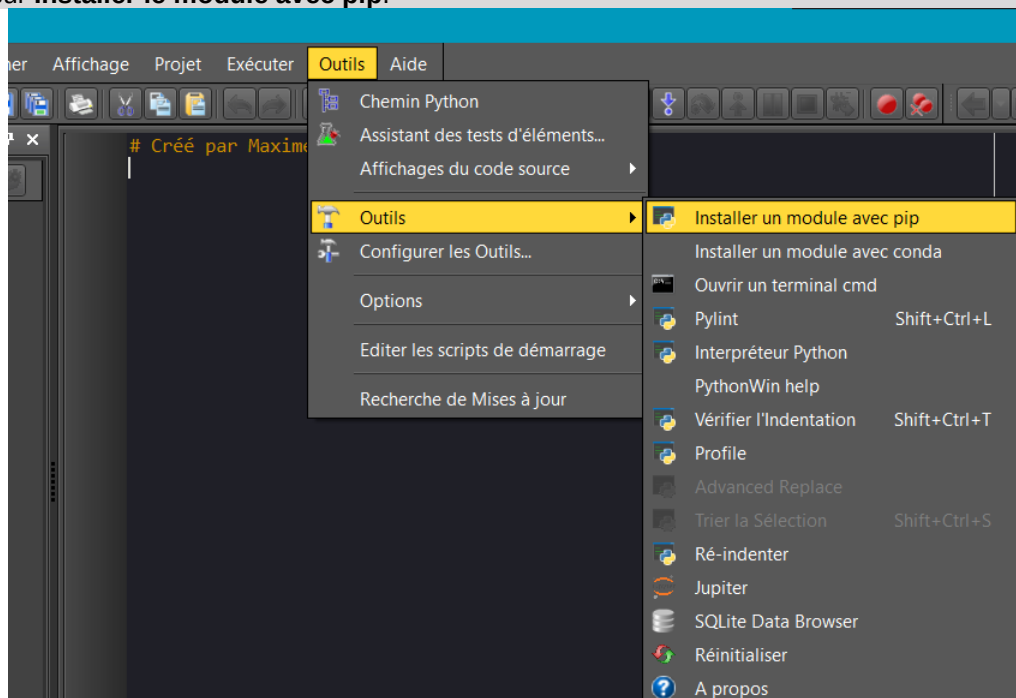
Les plus connus actuellement sont **Unity** et **Unreal Engine**.

a) Installation :

A FAIRE A LA MAISON !

Pyxel n'est pas installé par défaut sur vos ordinateurs, il y a une procédure à suivre pour installer la bibliothèque, détaillée étape par étape :

1. Commencez par ouvrir EduPython.
2. Dans la barre d'outils tout en haut, allez sur **Outils**.
3. Dans le menu, allez encore une fois sur **Outils**.
4. Cliquez sur **Installer le module avec pip**.



5. Une petite fenêtre s'ouvre et vous demande le nom du module : écrivez **pyxel**, puis validez.

6. Dans la console doit s'afficher : **Le processus "Installer un module avec pip" est terminé, code de sortie : 00000000.**

b) Fonctionnalités :

Pour utiliser Pyxel, il faut 3 instructions obligatoire ainsi que deux commandes dans l'ordre suivant :

```
import pyxel # Importation de la bibliothèque graphique pyxel

pyxel.init(longueur, largeur, title=" ")# Initialisation de la fenêtre : taille=(longueur et largeur), titre.

def update():# met à jour les frames (rafraichissement de l'image à l'écran).
    #remplir ici : c'est dedans qu'il faudra écrire les fonctionnalités de votre jeu

def draw(): #permet de dessiner à l'écran.
    #remplir ici : c'est dedans qu'il faudra écrire ce que vous souhaitez afficher à l'écran

pyxel.run(update, draw) # Exécution de la fenêtre de jeu en fonction de l'update et du draw.
```

Sans cette dernière ligne votre jeu ne pourra jamais s'afficher ni fonctionner.

c) Documentation :

Voici un listing par catégorie des principales fonctionnalités utilisables. Vous pourrez en retrouver une liste plus complète [ici](#).

Toutes ces fonctionnalités sont des méthodes donc pour les utiliser il faudra utiliser « objet ».fonctionnalité

d) Fonctionnalités système :

width, height

Donne la largeur et la hauteur de la fenêtre.

quit()

Permet de fermer la fenêtre et d'arrêter le programme (proprement).

e) Fonctionnalités ressources :

image(index).load(x,y,fichier)

Permet de charger jusqu'à **maximum 3 images (index =0, 1 ou 2)**, aux points de coordonnées (x,y).

load(fichier)

Permet de charger un fichier d'extension **.pyxres** depuis votre ordinateur (image, son, musique). Ce fichier est créé à partir d'un éditeur de pyxel, sur lequel vous pouvez vous-mêmes dessiner vos éléments et votre "terrain" de jeu.

Voir le fonctionnement ici : <https://github.com/kitaopyxel/blob/main/docs/README.fr.md#comment-cr%C3%A9er-une-ressource>

f) Fonctionnalités entrées :

mouse_x , mouse_y

Donne la position en x et y de la souris.

mouse_wheel

Donne la valeur de la molette de la souris.

btn(key)

Renvoie **True** si la touche key est appuyée, **False** sinon.

La liste des touches est disponible ici : https://github.com/kitaopyxel/blob/main/python/pyxel/_init_.pyi

`btnp(key, [hold], [repeat])`

La même chose que la fonction précédente, mais permet de renvoyer **True** ou **False** en fonction de si la touche est pressée selon un certain temps.

g) Fonctionnalités graphiques :

Pyxel utilise un outil appelé **palette** pour configurer un ensemble de couleur. Le moteur se veut représenter des jeux en 16 bits, il n'y a donc que 16 couleurs de disponibles (allant de 0 à 15). Vous pourrez cependant modifier vos couleurs.

```
#On met l'ensemble des couleurs dans une liste (palette)
palette = pyxel.colors.to_list()
#On remplace la couleur à l'indice couleur_a_changer par nouvelle_couleur
#nouvelle_couleur est une valeur HEXADÉCIMALE de la forme 0xFFFFFF
#FFFFFF est le code hexadécimal à modifier
palette[couleur_a_changer] = nouvelle_couleur
#On donne la palette de couleur au moteur de jeu
pyxel.colors.from_list(palette)
```

`cls(couleur)`

Remplace tout l'écran avec la couleur en paramètre. `cls(0)` met un écran noir.

`pal(couleur1, couleur2)`

Remplace sur l'image la couleur1 par la couleur2.

`pset(x, y, couleur)`

Dessine un pixel en fonction de la couleur en paramètre, au point de coordonnées (x,y).

`line(x1, y1, x2, y2, col)`

Dessine une ligne de couleur col de (x1, y1) à (x2, y2).

`rect(x, y, w, h, col)`

Dessine un rectangle de largeur w, de hauteur h et de couleur col à partir de (x, y).

`rectb(x, y, w, h, col)`

Dessine les contours d'un rectangle de largeur w, de hauteur h et de couleur col à partir de (x, y).

`circ(x, y, r, col)`

Dessine un cercle de rayon r et de couleur col à (x, y).

`circb(x, y, r, col)`

Dessine le contour d'un cercle de rayon r et de couleur col à (x, y).

`elli(x, y, w, h, col)`

Dessinez une ellipse de largeur w, de hauteur h et de couleur col à partir de (x, y).

`ellib(x, y, w, h, col)`

Dessinez le contour d'une ellipse de largeur w, de hauteur h et de couleur col à partir de (x, y).

`tri(x1, y1, x2, y2, x3, y3, col)`

Dessine un triangle avec les sommets (x1, y1), (x2, y2), (x3, y3) et de couleur col.

`trib(x1, y1, x2, y2, x3, y3, col)`

Dessine les contours d'un triangle avec les sommets (x1, y1), (x2, y2), (x3, y3) et de couleur col.

`fill(x, y, col)`

Dessine une ellipse de largeur w, de hauteur h et de couleur col à partir de (x, y).

`blt(x, y, img, u, v, w, h)`

Copie une image allant des points de coordonnées (u,v) jusqu'à (w,h) de l'image en paramètre (image chargée au préalable, valeur allant de 0 à 2, comme on ne peut charger que 3 images en même temps), et la colle au point de coordonnées (x,y).

Très utile pour répéter des images/dessins.

`text(x, y, chaine, couleur)`

Permet d'écrire une chaine de caractère (paramètre chaine) d'une certaine couleur, au point de coordonnées (x,y). Très utile pour répéter des images/dessins.

h) Fonctionnalités audios :

Vous pourrez ajouter du son grâce aux fonctions que vous trouverez ici:

<https://github.com/kitaopyxel/blob/main/docs/README.fr.md#audio>

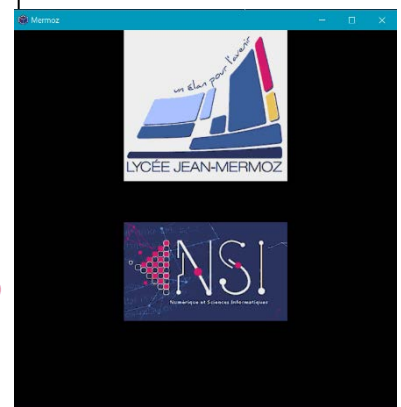
2. Prise en main :

Pyxel regorge d'exemples sur lesquels vous pouvez vous appuyer pour faire votre jeu. Les exemples sont disponibles [à cette adresse](#).

a) Ajout d'image :

Télécharger les images nécessaires ici : [S9_images.zip](#)

```
1 import pyxel
2 #Fenêtre avec pour titre "Mermoz"
3 pyxel.init(600, 600, title="Mermoz")
4 #image 255x236px depuis mon pc
5 pyxel.image(0).load(0,0,"logo_mermoz.png")
6 #image 255x154px depuis mon pc
7 pyxel.image(1).load(0,0,"NSI.png")
8 def update():
9     if pyxel.btnp(pyxel.KEY_Q):
10         pyxel.quit()
11 def draw():
12     pyxel.cls(0)
13     taille = 255 #maximum possible (pixels)
14     #charge l'image 0 (logo_mermoz), prend les pixels de (0,0) à (255,236)
15     #et la colle au centre
16     pyxel.blt(pyxel.width//2-(taille//2), 0, 0, 0, 0, taille, 236)
17     #charge l'image 1 (NSI), prend les pixels de (0,0) à (255,154)
18     #et la colle au centre
19     pyxel.blt(pyxel.width//2-(taille//2), 300, 1, 0, 0, taille, 154)
20 pyxel.run(update, draw)
```



Enregistrer sous images.py

b) Jouer avec les entrées :

```

1 import pyxel
2 #Fenêtre avec pour titre "Mermoz"
3 pyxel.init(600, 600, title="Mermoz")
4 #Chargement des images
5 pyxel.image(0).load(0,0,"NSI.png")
6 pyxel.image(1).load(0,0,"logo_mermoz.png")
7 change = 0 #Variable permettant de changer les images
8 def update():
9     #On définit en global pour que les modifications soient prises en compte
10    global change
11    if pyxel.btnp(pyxel.KEY_Q):
12        pyxel.quit()
13    #Appuyer sur N prendra l'image "0"
14    if pyxel.btnp(pyxel.KEY_N) :
15        change = 0
16    #Appuyer sur L prendra l'image "1"
17    if pyxel.btnp(pyxel.KEY_L) :
18        change = 1
19 def draw():
20    pyxel.cls(0)
21    #Dimension de l'image en fonction de "change"
22    taille_image = (pyxel.image(change).width,pyxel.image(change).height)
23    #Affichage de l'image
24    pyxel.blt(pyxel.width//2-(taille_image[0]//2), 0, change, 0, 0, taille_image[0], taille_image[1])
25 pyxel.run(update, draw)
26

```

Enregistrer sous images2.py

Que fais ce programme lorsqu'on appuie sur N ou L ou Q

REMARQUE :

Vous pourrez observer que ce programme utilise une **variable globale**. Celle-ci permet d'être utilisée dans toutes les fonctions, sans la passer en paramètre.

3. Exercices

Pour tous les exercices, on prendra une fenêtre de 300x300 pixels et le titre sera le nom du programme sans extension

Écrire un programme qui permet d'afficher toutes les couleurs de la palette dans un carré de taille 20px chacune, côte à côte.

Enregistrer sous couleurs.py

Écrire une fonction nsi() qui permet d'afficher grâce à des lignes les lettres « NSI » à l'écran. Cette fonction sera appelée dans la fonction draw() à la fin. Chaque lettre aura une hauteur de 100 pixels et une largeur de 40 pixels. L'espace entre chaque lettre sera de 10 pixels.

Enregistrer sous NSI.py

Pour les questions suivantes nécessitant d'utiliser des entrées, la liste des touches est disponible [ici](#), de la ligne 56 à la ligne 306. En les lisant, vous comprendrez ce qu'elles représentent !

Écrire un programme qui permet de déplacer latéralement un carré de 20 pixels de coté en appuyant sur la flèche gauche et droite. On utilisera une variable globale x. Le carré ne devra pas sortir de l'écran.

Enregistrer sous carre.py

Faire la même chose, mais en le déplaçant de haut en bas avec les flèches, et une variable globale y.

Enregistrer sous carre.py

Écrire un programme qui permet de dessiner des cercles ayant : une position x,y , un rayon r, et une couleur aléatoires (50<x<350 et 50<y<250 et 10<r<50 et 1<couleur<15 (on ne souhaite pas le noir)), en appuyant sur la barre d'espace. Vous utiliserez une liste.

Enregistrer sous cercle.py

Rajouter dans ce programme le fait de pouvoir supprimer aléatoirement un cercle en appuyant sur la touche supprimer.

Enregistrer sous cercle.py



Écrire un programme qui utilise une variable globale « couleur », et qui va dessiner un carré de taille 50px en fonction de la couleur. Ensuite, on changera la couleur en fonction du clic droit et gauche de la souris. Un clic droit signifie que l'on prend la couleur suivante dans la liste, clic gauche la couleur précédente.
Enregistrer sous `carre_couleur.py`

On va maintenant faire tomber la neige. Écrire une fonction `ajout_neige()` qui va générer des positions en x aléatoire, avec une taille de neige aléatoire, et ajoute ces données dans une liste globale. Pour faire tomber la neige périodiquement, vous pourrez utiliser : `pyxel.frame_count % un_nombre == 0`. Si `un_nombre = 30`, cela ajoutera à chaque seconde.
Enregistrer sous `neige.py`

Dans la fonction `update()`, appeler la fonction `ajout_neige()`, puis écrire le code nécessaire pour faire descendre la neige.
Enregistrer sous `neige.py`